

Card Suite

EUROBANK

Eurobank Installment Migration Script Migration

External specification

Table of Contents

1	Introduction	3
2	Purpose	4
3	Input File Columns	5
4	Due Date and Billing Date Logic	6
5	Error Handling	7
6	Logging and Progress	8

Disclaimer and notices

While every attempt has been made to ensure that the information in this document is accurate and complete, some typographical errors or technical inaccuracies may exist. *Tieto*evry does not accept responsibility for any kind of loss resulting from the use of information contained in this document.

The information contained in this document is subject to change without notice.

1 Introduction

This document describes the purpose, input requirements, and business logic of the

-)' 2 4% %"). 34 , , -%. 43 09 migration script used for *Eurobank* installment plan migration.

2 Purpose

The script migrates installment plans from a CSV file into *Card Suite*, creating the necessary agreements and posting installment entries according to account-level configuration.

3 Input File Columns

The input CSV file must contain the following columns:

COLUMN NAME	DESCRIPTION
##/5.4 ' 2%%-%.4 \$	The unique identifier for the account agreement to migrate.
).34 , ,-%.4 -/5.4	The total amount to be split into installments.
.5-/& .34 , ,-%.43	The number of installments for the plan – the rest of the installments.
, 34 .34 , ,-%.4	The final installment amount because it is different from others.
-%2#(.4 -%	The name of the merchant.

The script will fail if any of these columns are missing.

4 Due Date and Billing Date Logic

For each account agreement, the script retrieves 3 key values from the database:

Business Date Current business date of the system.

Billing Date The minimum value date from the account's CALA (Account Settlement Date Control) book account.

Due Day The due day is fetched from the account's effective conditions (condition index 102). The due date is then constructed using the billing date — billing date + due days.

In case when next due date < next billing date, it is handled with [redacted] event, using [redacted] - amount debited in previous period/billing cycle (the billed (invoiced) part), value date is current period end.

5 Error Handling

The script checks that the installment amount is evenly divisible by the number of installments. This takes into consideration the final installment. If not, the amount is rounded up from remainder if it is equal or higher to 5.

Example:

FIELD	VALUE
) . 34 , , -%. 4 -/5. 4	1000.00
. 5-/ & . 34 , , -%. 43	3
, 34 . 34 , , -%. 4	333.34

Last installment (provided) = **333.34** Regular installment = $(1000.00 - 333.34) / (\text{numofInstallments} - 1)$ = **333.33**
Total = $333.33 \times (\text{numofInstallments} - 1) + 333.34 =$ **1000.00**

The , 34 . 34 , , -%. 4 is part of) . 34 , , -%. 4 -/5. 4 — it is simply the final installment that absorbs any rounding difference.

Any failed rows are logged and written to a separate file for review.

6 Logging and Progress

The script logs all actions and errors to a log file. Progress is displayed in the console during migration.

Revision history

VERSION	DATE	CHAPTER	CHANGE
1.0.0	2026-07-17	All	Initial version
1.0.1	2026-07-17	Input Columns,	Added , 34 . 34 , , -%. 4 field
		Error Handling	